WEB ENGINEERING LAB

LAB 05

MARCH 4, 2026

TALHA NAVEED
01-131232-089

Lab # 5

Lab Task: Introduction to React with Vite

Lab Title

Getting Started with React – Components, JSX, and Props

Lab Objectives

By the end of this lab, students will be able to: - Set up a React project using Vite - Understand and use JSX syntax - Create and use functional components - Apply basic JSX rules - Pass data between components using props

---

Software Requirements

- Node.js (LTS version)
- npm (comes with Node.js)
- Visual Studio Code
- ES7+ React/Redux Snippets extension

Part A: Project Setup (React with Vite)

1. Create a new folder named **new**.
2. Open the **new** folder in **Visual Studio Code**.
3. Inside the **new** folder, create another folder named **frontend**.

Figure 1. Step 1- 3

1. Open the terminal and navigate to the frontend folder:

```
cd frontend
```

2. Check Node and npm versions:

```
node -v
npm -v
```

`npm will require administrative controls fo that run following:`

Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

```
PS C:\Users\Talha\Desktop\new> cd frontend
PS C:\Users\Talha\Desktop\new\frontend> node -v
v24.14.0
PS C:\Users\Talha\Desktop\new\frontend> npm -v
11.9.0
PS C:\Users\Talha\Desktop\new\frontend>
```

Figure 2. Step 4 & 5

3. Create a React project using Vite (initialize in the same folder):

   `npm create vite@latest .`

   - Select **React**
   - Select **JavaScript**
4. Install dependencies:

   `npm install`

5. Run the development server:

   `npm run dev`

```
PS C:\Users\Talha\Desktop\new\frontend> npm create vite@latest
Need to install the following packages:
create-vite@8.3.0
Ok to proceed? (y) y

> npx
> create-vite

◇ Project name:
  vite-project

◇ Select a framework:
  React

◇ Select a variant:
  JavaScript

◇ Use Vite 8 beta (Experimental)?:
  No

◇ Install with npm and start now?
  Yes

◇ Scaffolding project in C:\Users\Talha\Desktop\new\frontend\vite-project...

◇ Installing dependencies with npm...

added 157 packages, and audited 158 packages in 17s

33 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

◇ Starting dev server...
```

Figure 3. Step 6 – 8

6. Open the browser and verify the React app is running.

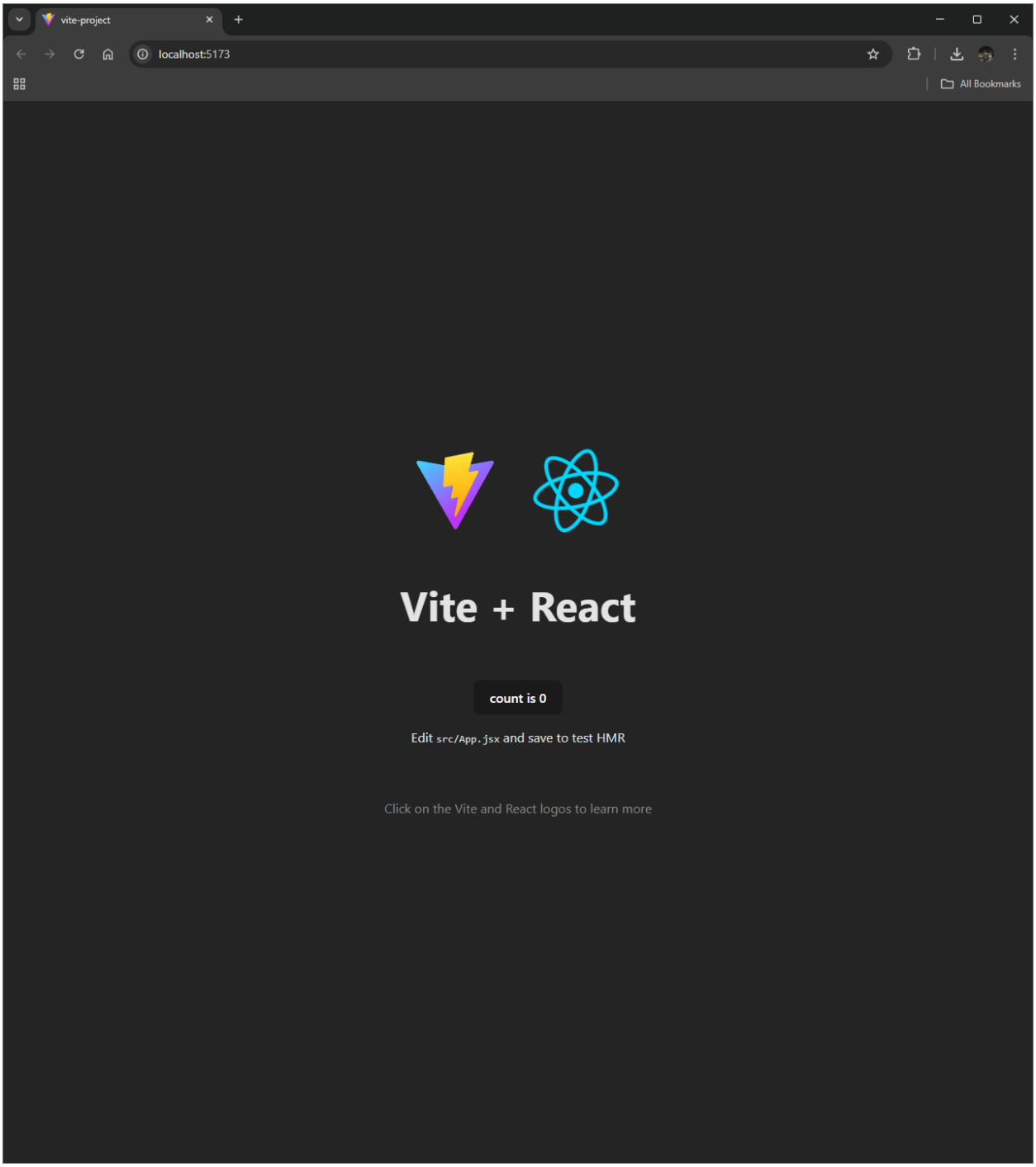
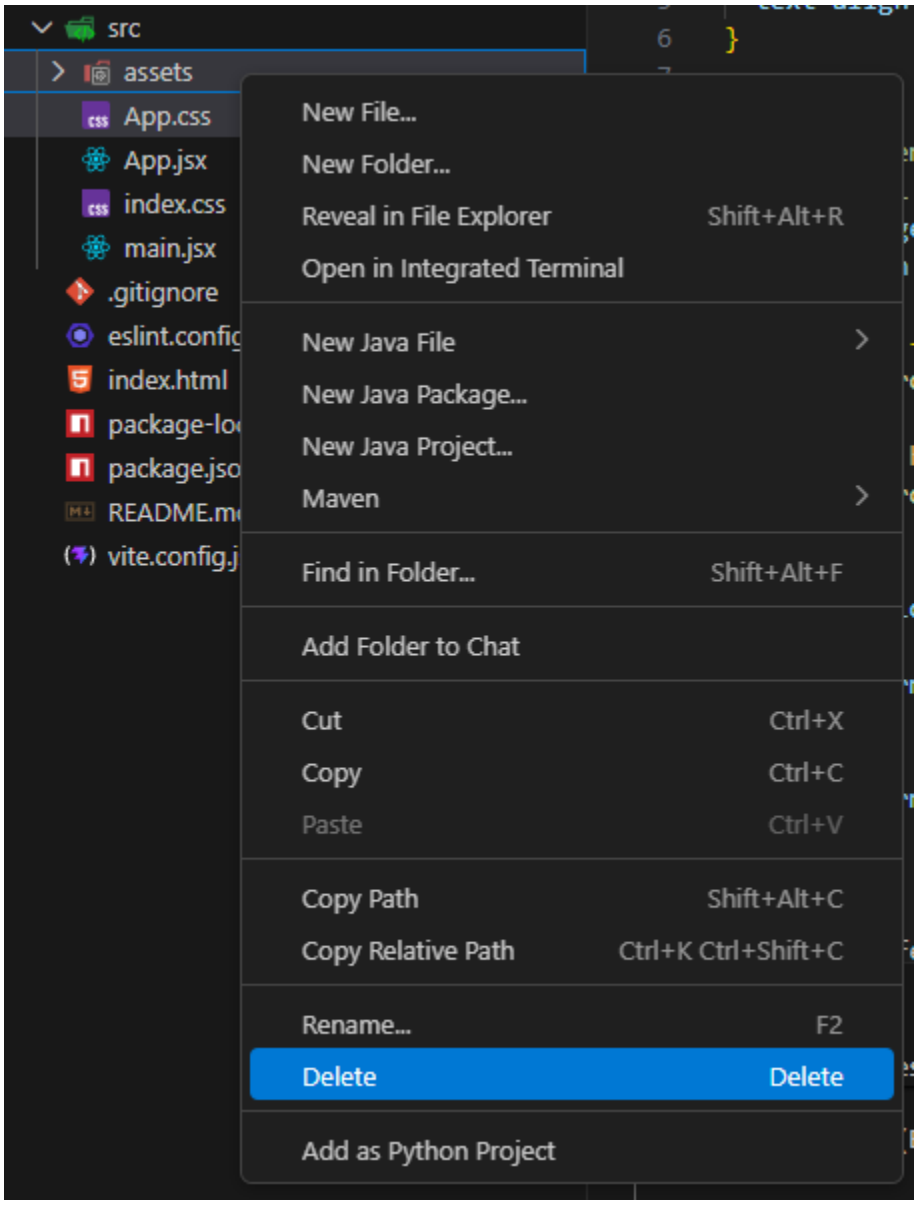


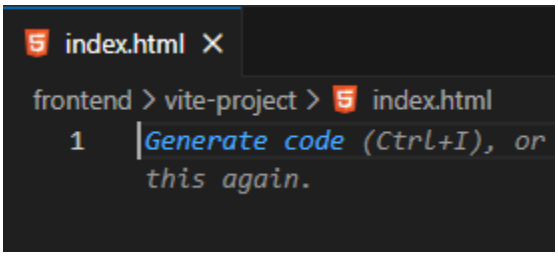
Figure 4. Verification Success

Part B: Project Cleanup

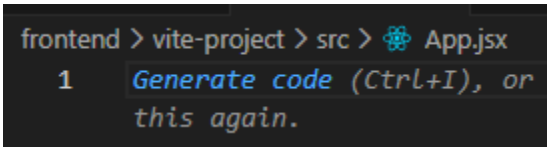
1. Delete the following:
    - **app.css** file
    - **assets** folder



2. Open **index.css** and delete all existing code.

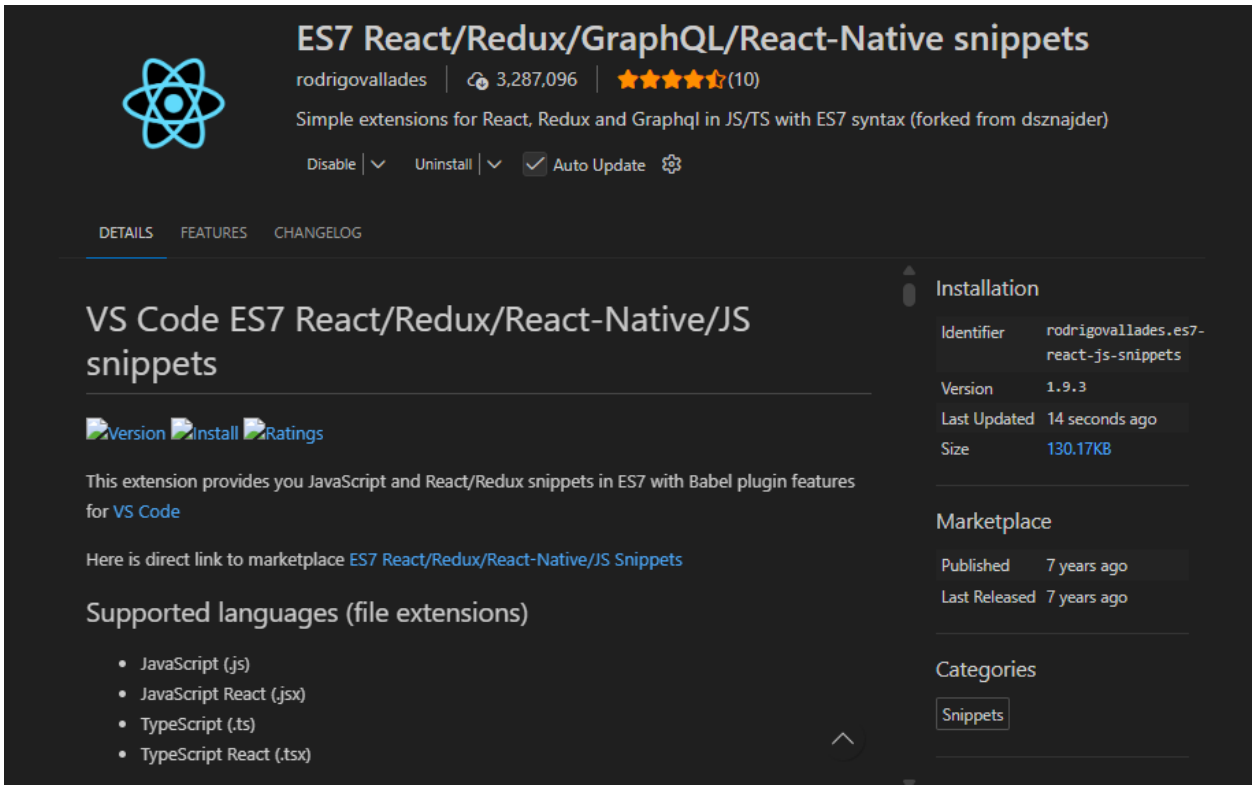


3. Open **App.jsx** and delete all existing code.



Part C: Creating a React Component Using rafce

1. Install the **ES7+ React/Redux Snippets** extension in VS Code.



2. Open **App.jsx**.

3. Type `rafce` and press **Enter** to auto-generate a functional component.

4. Modify the component to display the following message:

```
<h1>Welcome to React Lab</h1>
```

```
frontend > vite-project > src > App.jsx > ...
1   import React from 'react'
2
3   const App = () => {
4     return (
5       <div>
6         <h1>Welcome to React Lab</h1>
7       </div>
8     )
9   }
10
11  export default App
12
```

Note: - Component name must start with a **capital letter** - A component must return **JSX**

Part D: Understanding JSX

1. What is JSX?

JSX (JavaScript XML) allows writing HTML-like syntax inside JavaScript.

Example:

```
const element = <h1>Hello, React!</h1>;
```

---

2. JSX Rules

Rule 1: One Parent Element

```
return (
  <div>
    <h1>Hello</h1>
    <p>This works</p>
  </div>
);
```

Rule 2: Use `className` instead of `class`

```
<h1 className="title">Hello</h1>
```

Rule 3: JavaScript inside JSX using { }

```
const name = "Alice";

function Greeting() {
  return <h2>Hello, {name}!</h2>;
}
```

Part E: Creating and Using Components

1. Simple Functional Component

Create a new file **Welcome.jsx**:

```
function Welcome() {
  return <h2>Welcome Component</h2>;
}

export default Welcome;
```

2. Using a Component in App.jsx

```
import Welcome from './Welcome';

function App() {
  return (
    <div>
      <Welcome />
    </div>
  );
}

export default App;
```

Part F: Props (Passing Data to Components)

Component with Props

Update **Welcome.jsx**:

```
function Welcome(props) {
  return <h2>Hello, {props.name}!</h2>;
}

export default Welcome;
```

Passing Props from App.jsx

```
<Welcome name="Student" />
```

Part G: Component Types (Theory)

Class Component (Example)

```
import React, { Component } from 'react';

class Hello extends Component {
  render() {
    return <h2>Hello from Class Component</h2>;
  }
}

export default Hello;
```

Functional Component (Recommended)

```
function Hello() {
  return <h2>Hello from Functional Component</h2>;
}

export default Hello;
```

> **Note:** Functional components are the modern and recommended approach in React.

Lab Tasks / Exercises

1. Create a new component named **Student.jsx** that displays student name and course.

Code for Student.jsx:

```
function Student(props) {

  return (
    <div className="student-container">
      <h2 className="student-heading">Student Name: {props.name}</h2>
      <h3 className="course-heading">Course: {props.course}</h3>
    </div>
  );
}

export default Student;
```

2. Pass student data from **App.jsx** using props.

Code for App.jsx:

```
import Student from './Student';


function App() {
  return (
    <div>
      <h1>Lab 5: React Props</h1>
      <Student name="Talha Naveed" course="Software Engineering (BSE-6A)" />
    </div>
  );
}

export default App;
```

3. Apply `className` to style headings.

```
function Student(props) {
  return (
    <div className="student-container">
      <h2 className="student-heading">Student Name: {props.name}</h2>
      <h3 className="course-heading">Course: {props.course}</h3>
    </div>
  );
}

export default Student;
```

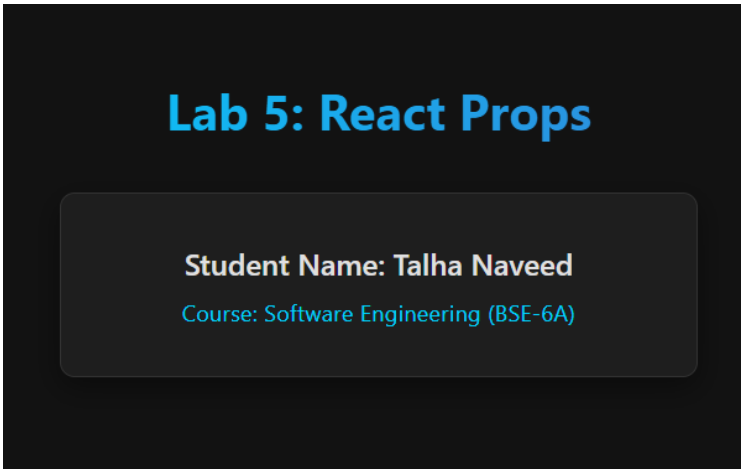


Figure 5. Output

Source Code Link: ***https://github.com/TALHA-089/WEB-LAB-REACT.git***